

Optimization and Stability Enhancements in Gambit’s Quantal Response Equilibrium (QRE) Tracing Algorithm

Andrés Fernández Cervell

May 20, 2026

Abstract

This report details the empirical validation, debugging, and optimization of the logit-based Quantal Response Equilibrium (QRE) tracing algorithm in Gambit. A grid search was conducted across 196 test games to evaluate the baseline performance versus a modified algorithm featuring symmetry-breaking perturbations. The modified engine successfully resolved previously failing games, including `team_mp.txt`. Furthermore, an architectural bug related to degenerate game trees was identified and patched, and error-handling mechanisms were overhauled for robustness. More details about the first draft or the proposed modifications can be found in Solving Issue #492.

1 Methodology and Baseline Evaluation

To rigorously evaluate the stability of the tracing algorithm, a 2D Grid Search was executed across the entire Gambit test suite, comprising 196 games located in the `contrib/games` and `tests/test_games` directories.

The evaluation tested the algorithm’s sensitivity to path-following dynamics by varying two core parameters:

- **Initial Step Size (S):** Tested across 3 different values: 0.1, 0.01, 0.03.
- **Maximum Acceleration (A):** Tested across 3 different values (comparing default settings with nearby optimal candidates): 1.05, 1.1, 1.5.

A test was marked as a “failure” if the algorithm terminated without converging to a valid Nash Equilibrium (NE). It is important to note that while some games in the test suite are structurally designed not to yield an NE under this specific solver (e.g., games with imperfect recall), others failed due to mathematical divergence.

As shown in the baseline results (Figure 1), the best baseline configuration yielded **166 successes and 30 failures**.

FINAL RESULTS:		
PARAMETERS	SUCCESS	FAILURE
S:0.1 A:1.05	165	31
S:0.03 A:1.5	165	31
S:0.03 A:1.1	165	31
S:0.01 A:1.05	164	32
S:0.1 A:1.5	164	32
S:0.1 A:1.1	166	30
S:0.03 A:1.05	164	32
S:0.01 A:1.5	166	30
S:0.01 A:1.1	165	31
🏆 BEST CONFIGURATION: S:0.1 A:1.1		
With a total of 166 successes and 30 failures.		

Figure 1: Baseline Grid Search Results: 166 Successes, 30 Failures.

2 Modified Algorithm Results

The same 196 games were re-evaluated under the identical parameter grid using the modifications of the first draft.

The results demonstrated a significant improvement in convergence stability. For the optimal parameter combination, the modified algorithm achieved **172 successes and 24 failures**. This represents a successful resolution of 6 previously failing test games, in addition to resolving `team.mp.txt`, which was the primary object of this study.

FINAL RESULTS:		
PARAMETERS	SUCCESS	FAILURE
S:0.1 A:1.05	171	25
S:0.03 A:1.5	171	25
S:0.03 A:1.1	170	26
S:0.01 A:1.05	170	26
S:0.1 A:1.5	170	26
S:0.1 A:1.1	172	24
S:0.03 A:1.05	170	26
S:0.01 A:1.5	171	25
S:0.01 A:1.1	170	26
🏆 BEST CONFIGURATION: S:0.1 A:1.1		
With a total of 172 successes and 24 failures.		

Figure 2: Re-evaluation Search Results: 172 Successes, 24 Failures

3 Failure Analysis

An in-depth analysis was conducted on the 24 remaining failures in the optimal configuration (Figure 3).

```

=====
Games analyzed: 196
Incidents by WARNING: 4
Incidents by LIMIT/OTHER: 20
=====
>>> DETAIL OF INCIDENTS BY WARNING (Bifurcations/Errors):
TEST: ./contrib/games/bayes2a.efg
WARNING: Logit solver stopped at lambda = 359688 with regret = 6.42359e-07 (Goal was regret = 2e-07)
Best profile found = 359688.269807,0.555556,0.444444,0.4,0.6,0.4,0.6,1,0,1,0,0.111111,0.888889,0,1,0,1,0,1,0.5,0.5,0.333333,0.666667,0.333333,0.666667,0.533333,0.466667,0.5,0.5,0.333333,0.666667,0.333333,0.666667,0.533333,0.466667,0.533333,0.466667
-----
TEST: ./contrib/games/bcp4.efg
WARNING: Logit solver stopped at lambda = 372.22 with regret = 0.000738337 (Goal was regret = 4e-08)
Best profile found = 372.220036,7.43849e-163,4.94066e-324,1,2.22276e-162,0,1,0.249265,0.750735,9.88131e-324,0,1,2.22276e-162,1,2.22276e-162,4.94066e-324
-----
TEST: ./contrib/games/caro2.efg
WARNING: Logit solver stopped at lambda = 4.87755e+06 with regret = 5.68901e-08 (Goal was regret = 4e-08)
Best profile found = 4877550.824292,0.605219,0.394781,1,0,0,1,0.535073,0.464927,0.20174,0.79826,0,1,0.333334,0.666666,0,1,0,1,0.333333,0.666667,1,0,1,0
-----
TEST: ./contrib/games/vd.efg
WARNING: Logit solver stopped at lambda = 675.078 with regret = 0.000406705 (Goal was regret = 4e-08)
Best profile found = 675.077641,6.5691e-294,1,0.749594,0.250406,0.749594,0.250406,0.250406,0.749594,0.250406,0.749594,0.250406,0.749594
=====

```

Figure 3: Detailed trace of the 24 unresolved games in the optimal configuration.

These 24 anomalies were classified into two distinct categories:

1. **Structural Errors (20 games):** These games failed gracefully via an **ERROR** message. These are not algorithmic failures; rather, these specific games contain properties (such as imperfect recall) that are fundamentally unsupported by the current QRE tracing solver by design.
2. **Algorithmic Warnings (4 games):** These represent genuine mathematical tracing failures where the algorithm threw a **WARNING** due to bifurcation traps or tolerance limits.

3.1 Analysis of Algorithmic Warnings

Among the 4 genuine failures, `bayes2a.efg` fails due to highly complex symmetric breaking requirements discussed in previous analyses: After 17 seconds of execution time, the regret is almost the one expected. Since it is a very complex game to compute, the numbers end up being mostly noise, so the algorithm does not reach the expected regret by a very little margin, still being a very precise approximation.

The remaining 3 games fail due to limitations inherent to the **IEEE 754 double-precision floating-point format**. As the path-follower approaches the equilibrium at extremely high λ values, the variables representing probabilities and step corrections approach infinitely small numbers. When these values drop below the minimum representable threshold, C++ experiences floating-point underflow. The numbers evaluate to pure zero, causing a sudden explosion of the numerical method.

4 Debugging the Degenerate Game Bug

While investigating the 20 structural errors, a critical, unhandled C++ exception was discovered natively in the Gambit codebase:

```
what(): vector::_M_range_check: __n (which is 0) >= this->size() (which is 0)
```

4.1 Root Cause Analysis

The crash originated in `logit.cc` when processing *degenerate games* (e.g., `chance_root_5_moves_no_nonterm_player_nodes.efg`). This specific game consists entirely of Nature (chance) nodes and terminal nodes, with absolutely zero decision nodes

for human players. Consequently, the `Information Sets` array is initialized with a size of 0. When the mathematical engine subsequently attempted to access the first element, the operating system triggered a segmentation fault.

4.2 Proposed Solution

To prevent the engine from processing nonsensical matrices, a defensive programming shield was introduced in `logit.cc` immediately after loading the game. The patch checks if the player's information sets collection has a size greater than zero before proceeding:

```

1  if (game->IsTree()) {
2      bool has_decisions = false;
3      for (auto player : game->GetPlayers()) {
4          if (player->GetInfosets().size() > 0 && !has_decisions) {
5              has_decisions = true;
6          }
7      }
8      if (!has_decisions) {
9          std::cout << "ERROR: Game contains no player decision nodes (chance only). No equilibria to compute.\n";
10         return 0;
11     }
12 }

```

Listing 1: Defensive patch in `logit.cc` to handle degenerate trees.

5 System Default Updates and Error Handling

Based on the empirical data gathered during the Grid Search, the default engine parameters were permanently updated to reflect the discovered global optimum, ensuring maximum robustness out-of-the-box:

- **Initial Step (S):** Updated from 0.03 to 0.1.
- **Maximum Acceleration (A):** Hardcoded default set to 1.1.

Finally, the error handling pipeline was refactored to avoid the `exit(1)` code when convergence fails. In `efglogit` and `nfglogit`, if the target regret is not met (triggering a WARNING), the function now returns an empty profile list. In `logit.cc`, a safety check verifies that the returned array is not empty before attempting to print the profile:

```

1  auto result = LogitBehaviorSolve(start, maxregret, 1.0, hStart,
2      maxDecel, printer);
3  if (!result.empty()) {
4      PrintProfile(std::cout, decimals, result.back(), true);
5  }

```

Listing 2: Safe profile printing in `logit.cc`

This prevents undefined behavior and ensures that scripts relying on the standard output only receive valid Nash Equilibria.

6 Conclusion

The stability of Gambit's QRE tracing algorithm has been considerably increased thanks to the modifications proposed in Solving Issue #492. Furthermore, the identification and resolution of the `chance_root_5_moves_no_nonterm_player_nodes.efg` bug and the implementation of strict output validation ensure that the software handles edge cases safely and predictably.